Approach: Query-Adaptive Retrieval and Evidence Verification

A reading guide for the concept and technical design. Everything here describes what the code in `backend/app` does. **No experimental results are stated in this document.** Results come only from the evaluation runs and are reported in `experiments/results/`.

---

1. The problem in one page

A conventional RAG system does: *embed question fetch top-k chunks (k fixed, e.g. 5) generate answer*. Two weaknesses motivate this project.

Weakness	What goes wrong
Fixed k ignores the question	A one-fact question gets 5 chunks (noise, tokens, cost). A mul...
Retrieval != truth	Even with relevant chunks in the prompt, an LLM can add uns...

Two contributions

1. **Query-adaptive retrieval.** Estimate how complex the question is, then choose how many chunks to retrieve (and whether to search per sub-question).
2. **Evidence verification.** Split the answer into atomic claims, check each claim against its cited passage with an entailment model, and if something is unsupported, retrieve more evidence, regenerate,
   and re-verify (bounded retries). If still unsupported, say so instead of presenting it as fact.

The research claim is only that these two together are *worth testing against* fixed top-k. Whether they win is an empirical question answered by the experiments, not assumed.

---

2. System at a glance

```
                    React UI   REST/SSE            FastAPI (backend/app)
```

Module map:

Folder	Role
ingestion/	load PDF/DOCX/TXT/MD clean chunk embed store; docum...
vectorstore/	VectorStore protocol + ChromaDB implementation (cosine spa...
retrieval/	dense.py (shared), baseline_retriever.py, complexity_classifier...
generation/	the single generation prompt, generator, citation parsing
verification/	claim_extractor, evidence_matcher, nli, verifier, citation_verifie...
workflows/	baseline_pipeline.py, rag_graph.py (LangGraph), query_servic...
core/	settings + research config, LLM factory, structured JSON calls,...

evaluation/ + experiments/	metrics, benchmark runner, results

3. Ingestion

1. **Load**: PDF via pypdf (keeps 1-based page numbers), DOCX via docx2txt, TXT/Markdown with UTF-8 CP1252 fallback. Files with no extractable text are rejected (no OCR).
2. **Clean**: NFKC normalisation (expands ligatures), control characters removed, Markdown syntax stripped; for PDFs, de-hyphenation and re-joining of hard-wrapped lines.
3. **Chunk**: recursive character splitter preferring paragraph line sentence boundaries.
   Default **1000 chars / 150 overlap**, configurable. **Pages are chunked separately**, so each chunk maps to exactly one page.
4. **Metadata** on every chunk: `chunk_id` (`docid:00012`), `doc_id`, `filename`, `page`, `chunk_index`, `char_start`. This is what makes citations resolvable to a source location.
5. **Embed and store**: embeddings computed outside the store (same embedding model for every pipeline), upserted into one Chroma collection per corpus.

The document registry also stores the file SHA-256 (duplicate uploads rejected per corpus), the status (`processing / ready / failed` with an error message) and the chunk settings used.

Demo corpus. On first start (with an API key) the backend indexes three synthetic documents from `backend/seed/`: `institute_overview.md`, `research_labs.md`, `graduate_regulations.pdf`. They describe a
fictional institute, so an LLM cannot know the facts from pre-training. A marker file prevents re-seeding after the user deletes documents.

---

4. The baseline

`question embed top-k (default 5) numbered context [1..k] generate parse citations`

- Same `DenseRetriever`, same embedding model, same chat model, same generation prompt as the proposed
  system. Only the retrieval policy and the absence of verification differ.
- The prompt (`generation/prompts.py`) tells the model to answer only from numbered passages, cite each
  factual sentence as `[n]`, and say so when passages are insufficient.
- Citation parsing splits the answer into sentences, attaches the `[n]` markers to their sentence, maps numbers to chunk IDs, and reports numbers matching no passage as `invalid_citations`. Abbreviations ("Dr.", initials) do not end a sentence.

Fairness by construction

```
shared                          differs        vector store + corpus
```

A test asserts that the proposed graph with `adaptive=False, verify=False` retrieves the same chunks and

sends the byte-identical generation prompt as the baseline class. That is how we know the ablation variants isolate the effect of each component.

---

5. Contribution 1: query-adaptive retrieval

5.1 Complexity assessment (`retrieval/complexity_classifier.py`)

A hybrid of an LLM analysis and cheap deterministic features. **Query length is not a feature.**

Step 1: LLM analysis (one JSON call, temperature 0) returns:

Field	Meaning
information_needs	the distinct facts that must be looked up, as standalone sub-q...
reasoning_type	lookup / aggregation / comparison / temporal / causal / bridge
requires_multiple_sources	facts probably live in different documents
complexity	the LLM's own label: simple / moderate / complex
rationale	one sentence

The prompt tells the model that referring to an entity by description ("the lab that runs Project X") creates a separate information need, because that entity must be found first (a *bridge*).

Step 2: lexical features (no LLM): comparison markers ("versus", "differ", "which ... or ..."), enumeration markers, nested relation markers ("of the", "'s", relative pronouns), entity/date mentions.

```
lexical = 0.35·[comparison>=1] + 0.25·min(enum/2,1) + 0.25·min(nesting/3,1) + 0.15·min((entities1
```

Step 3: combine into one score in [0, 1]:

```
S = ( 0.35·needs + 0.25·reasoning + 0.15·multi_source + 0.15·lexical + 0.10·llm_label ) /  weight
```

All weights, reasoning scores and thresholds live in `ComplexityConfig`; they are meant to be tuned on a **dev split**, never on the test split.

Step 4: label and depth.

Label	Score range	k band
simple	S < 0.33	1 - 3
moderate	0.33 <= S < 0.66	4 - 7
complex	S >= 0.66	8 - 12 (hard cap k_cap = 15)

k is **interpolated inside the band** by where S falls in the label's score range:

```
position = (S  range_low) / (range_high  range_low)      clamped to [0,1]k        = band.min + ro
```

So S = 0.34 (barely moderate) gives k = 4; S = 0.65 gives k = 7. Depth varies smoothly rather than in three buckets.

Failure handling. If the LLM reply is not valid JSON after one repair attempt, the system falls back to the lexical score alone and flags `fallback=true`. The pipeline never crashes on a classifier error.

5.2 Decomposition and fusion (`retrieval/adaptive_retriever.py`)

If there are at least two information needs (and `decompose` is on), the system searches for the original question **and** each sub-question (max 4), each at depth k. The ranked lists are merged with **Reciprocal Rank Fusion**:

```
RRF(chunk) =  over queries  1 / (60 + rank_of_chunk_in_that_query)
```

RRF needs no score calibration across queries. A **coverage guarantee** is added: each query's best chunk is placed in the final selection first, then the remaining slots are filled by RRF order. This stops a middle-hop chunk, which ranks low for the full question, from being crowded out. Each selected chunk records which query retrieved it (`original_query` or `sub_query:i`).

Why this matters for multi-hop: "Which building houses the lab that runs Project Veritas?" needs *(a)* the lab that runs Veritas and *(b)* that lab's location. Plain top-k on the full question may return only (a). A sub-question search for (b) fetches the second hop.

5.3 Retrieval confidence and expansion

After retrieval the system estimates whether the evidence is likely sufficient:

```
top1     = normalise(best cosine)          normalise(x) = clamp((x  0.15) / (0.65  0.15), 0, 1)me
```

If `C < 0.5` and expansions remain (`max_expansions` = 1, `k_cap` not reached), **expansion** raises k by `expansion_step` (3, capped) and re-searches the *uncovered* queries (or all, if none is uncovered), then re-fuses and re-assesses. The floor/ceiling/threshold values are initial defaults to be calibrated on the dev split.

Logged per query: original question, label, score, selected k, final k, number retrieved, expansions, confidence, retrieval latency (query embedding + vector search).

---

6. Contribution 2: evidence verification

6.1 Claim extraction (`verification/claim_extractor.py`)

The LLM turns the answer's sentences into **atomic, self-contained claims** (pronouns resolved, one fact each). Two design choices make this trustworthy:

- **Citations are not taken from the LLM.** Each claim inherits the `[n]` markers actually present in its source sentence, so traceability is anchored in the answer text.
- **Abstentions are not claims.** "The passages do not contain the budget" is detected (a strong pattern, or a weak negation plus a reference to the sources) and skipped, because it asserts nothing to verify. "He was *not* elected" is still a claim.

If extraction fails, fallback is one claim per sentence (`extraction_fallback=true`).

6.2 Evidence windows (`verification/evidence_matcher.py`)

Each retrieved chunk is split into overlapping windows of **3 sentences**. Claim and window embeddings are compared by cosine similarity, and the top **3 windows** per claim become candidates. Embeddings are

cached across attempts of one query.

> Similarity only decides *what to test*. It never decides whether a claim is supported. A sentence can be very similar to a claim and still contradict it ("founded in 1999" vs "founded in 2016").

6.3 Entailment (`verification/nli.py`, `verifier.py`)

A local NLI cross-encoder, default `MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli`, runs on CPU. It gives
P(entailment), P(neutral), P(contradiction) for (premise = evidence, hypothesis = claim). Label order differs between checkpoints, so indices are read from each model's `id2label`.

For each claim, NLI tests: the candidate windows, the **whole passages** those windows came from (a window can lose the entity a passage is about, such as "the lab"), and the **two best passages combined** (claims joining two facts).

6.4 Decision procedure

```
claim  test windows of the CITED passages          entailment >= 0.75  SUPPORTED   (citation co
```

Setting	Default
support_threshold	0.75
contradiction_threshold	0.75
partial_threshold	0.40 (used only when the judge is disabled)
window_sentences / candidate_windows	3 / 3

6.5 LLM judge with quote verification

Only NLI-ambiguous claims reach the judge (keeps cost down). The judge sees up to 4 passages (cited ones
first, then best-matching), and returns:

```
{"verdict": "supported | partial | unsupported", "passage": 2, "evidence_quote": "<copied from th
```

Quote check (`citation_verifier.quote_in_passage`): the quote must actually occur in the named passage (>= 85 % match after normalisation, minimum length). If it does not, a "supported" or "partial" verdict is **downgraded to unsupported**. This blocks a judge that "supports" a claim with evidence it made up. If the judge's output cannot be parsed, the claim is marked unsupported (fail closed).

With `use_llm_judge=false`, ambiguous claims are decided by threshold: entailment >= 0.40 partial, else
unsupported.

6.6 Citation status

Status	Meaning
correct	a cited passage supports the claim
misattributed	cited passages do not, but a different retrieved passage does ...

missing	the claim carries no valid citation
incorrect	it cites passages and nothing retrieved supports it

`citation_accuracy` = share of cited claims whose status is `correct`.

6.7 Scores and the pass rule

```
support_score = (#supported + 0.5·#partial) / #claimspassed       = support_score >= 0.90  AND
```

An answer with no factual claims (an honest "the documents don't say") is `abstained` and passes.

---

7. The LangGraph workflow (`workflows/rag_graph.py`)

```
START  adaptive?  yes  analyze_query  select_depth  retrieve  assess_confidence
```

The graph state holds the question, config, complexity, k, chunks, confidence, attempts, claims, refinements, usage and per-node timings. Two flags in `PipelineConfig.proposed` - `adaptive` and `verify` - produce **every ablation variant from this one graph**. A recursion limit guards against runaway loops.

7.1 Refinement (`_refine`)

When verification fails and retries remain (`max_retries`, default 2):

- **No-evidence claims** (and partial ones) become search queries. For each, up to `refine_k` (3) **new** chunks are fetched (chunks already seen are excluded) and appended with new citation numbers.
- **Contradicted claims** trigger no retrieval, because the evidence is already there. They produce feedback naming the conflicting passage.
- A **revision note** (list of problem claims, suggested citations, new passage range) is appended to the *same* generation prompt, along with the previous answer. Then the loop regenerates and re-verifies.

7.2 Finalisation and uncertainty (`verification/finalize.py`)

- **Best attempt**, not simply the last: ranked by *passed*, then higher support score, then fewer unsupported claims, then latest.
- A sentence whose claims are all unsupported is **removed from the final answer**. A sentence with mixed
  claims is kept and marked partial. If nothing supported remains, the answer is replaced by:
  "The retrieved documents do not provide enough verified evidence to answer this question."
- An `Uncertainty` object carries a plain-language notice (how many statements could not be verified and were removed, how many are only partial), the unsupported claims, and the removed sentences. The UI shows these. **Unsupported claims are never presented as fact.**

---

8. A worked example (illustrative design trace)

Question over the demo corpus: *"Which building houses the lab that runs Project Veritas?"*

1. **Analysis.** The description "the lab that runs Project Veritas" must be resolved first, so the LLM should list two needs: *which lab runs Project Veritas?* and *where is that lab located?* - reasoning type `bridge`. The score S lands in the moderate or complex band, and k is chosen inside it. (If the classifier judges it a single need, k stays small; that is exactly the behaviour the dev-split calibration and the classifier-accuracy metric are meant to examine.)
2. **Retrieval.** Three searches (original + 2 sub-questions), RRF fusion with the coverage guarantee. The chunk about "Project Veritas is run by the Retrieval and Reasoning Lab" and the chunk "located on the third floor of Aster Hall" are the targets.
3. **Confidence.** C is computed. If low, one expansion deepens the search.
4. **Generation.** The shared prompt produces a cited answer.
5. **Claim extraction.** E.g. *"The Retrieval and Reasoning Lab runs Project Veritas"* and *"The Retrieval and Reasoning Lab is located on the third floor of Aster Hall"*.
6. **Verification.** Each claim is NLI-tested against its cited windows. A claim NLI cannot settle goes to the judge, whose quote must exist in the passage.
7. **Outcome.** All claims supported and no unsupported return. Otherwise refine, regenerate, re-verify, up to 2 retries; return the best attempt with any uncertainty flagged.

Other demo questions probe other behaviour: a simple lookup (founding year), a comparison with arithmetic (which lab is older and by how many years), a multi-hop with the regulations PDF (who appoints the third examiner), and an **unanswerable** question (library budget) where the correct behaviour is to abstain rather than guess.

---

9. Experimental design

9.1 Configurations (the 2*2 ablation plus sweeps)

Name	Retrieval	Decompose	Verification + retry
B baseline	fixed k = 5	-	-
A adaptive only	adaptive	yes	-
A adaptive, no decomposition...	adaptive	no	-
V verification only	fixed k = 5	-	yes
A+V full system	adaptive	yes	yes
K-sweep	k {1,2,3,5,8,10,15}...	-	-
R-sweep	A+V with retries {0..3}...	yes	yes

Mapping to the five experiments: (1) B vs A; (2) with vs without verification (A vs A+V, B vs V); (3) K-sweep per complexity group; (4) R-sweep; (5) factual reliability vs cost/latency across all configurations.

9.2 Data and splits

MuSiQue (human-annotated supporting paragraphs; known hop counts): simple single-hop, moderate 2-hop,
complex 3-4-hop, plus optional unanswerable questions. A **dev split** is used for all calibration (weights, thresholds, score normalisation); a held-out **test split** produces all reported numbers. The gold and distractor paragraphs of the sampled questions form one shared corpus.

9.3 Metrics

Group	Metrics
Retrieval	Precision@k, Recall@k (at each system's own k and fixed cut...
Answer	exact match, token F1 vs gold + aliases, RAGAS correctness /...
Evidence	claim support rate, unsupported-claim rate, citation precision/r...
Efficiency	chunks per query, retrieval and total latency, tokens, estimated...
Classifier	accuracy and confusion matrix vs hop-count labels

Statistics: paired comparisons on identical questions, bootstrap 95 % confidence intervals, McNemar / Wilcoxon tests. Fixed seed, temperature 0, recorded config snapshot per run.

---

10. Evaluation independence and honest limitations

The circularity risk. If the system's own verifier also scores the results, the proposed system wins by construction. Mitigations:

- the evaluator uses a **different NLI checkpoint** (`cross-encoder/nli-roberta-base`, setting `eval_nli_model`) than the verifier (DeBERTa);
- a **stronger, different judge model** (`judge_model`, default `gpt-4.1`) scores faithfulness and correctness, and is never used inside the evaluated pipelines;
- a **human-labelled claim sample** with inter-annotator agreement (Cohen's) measures how far the automatic judges can be trusted. Automatic and human-verified results are reported separately. If no human labels exist, that gap is stated, not filled in.

Known limitations (state these in the report):

Limitation	Why it matters
Classification adds an LLM call to every query	Savings from smaller k can be outweighed by extra tokens an...
In the current wiring, the in-pipeline judge, claim extractor and ...	They may share blind spots; the independent evaluator exists ...
Cosine scores are weakly calibrated	The confidence signal is noisy; thresholds are only as good as...
NLI is weak on numbers, dates and multi-step arithmetic	Such claims lean on the judge; error rates are checked on the ...
Gold labels may be incomplete	A distractor paragraph may also answer a question, understati...
Small per-group test sets	Wide confidence intervals; report intervals, not just means
Removing unsupported sentences can lower completeness	Verification trades recall of content for reliability
Scanned PDFs unsupported	No OCR
Pricing table is hard-coded defaults	Verify against current prices before quoting costs

---

11. Related work (to cite and position against)

Work	Relation to this project
Adaptive-RAG (Jeong et al., NAACL 2024)	Already routes queries by a simple/moderate/complex classifie...
Self-RAG (Asai et al., 2023)	Trains the LLM to emit critique tokens. Ours uses external, ins...

CRAG (Yan et al., 2024)	Evaluates retrieval quality and corrects it. Ours judges the ans...
FActScore (Min et al., 2023)	Atomic-fact decomposition for scoring. We borrow the claim-s...
ALCE (Gao et al., 2023)	NLI-based citation precision/recall. Basis for our citation metric...
RARR (Gao et al., 2023)	Post-hoc revision to attribute output. Ours is similar in spirit, wi...

Honest positioning: neither idea is new in isolation. The contribution under test is the **combination**, evaluated in a controlled ablation.

---

12. Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation: retrieve text, then generate ...
top-k	the k highest-scoring chunks returned by the vector search
Embedding	a vector representing text meaning; similar texts have high co...
Chunk	a passage-sized piece of a document, the unit of retrieval and ...
Multi-hop	a question needing facts from several passages chained toget...
Information need	one distinct fact that must be looked up
Bridge	a hop whose answer is needed before the next lookup can be ...
RRF	Reciprocal Rank Fusion: merges ranked lists by summing 1/(c...
Claim	one atomic factual statement extracted from the answer
NLI / entailment	judging whether a premise implies, is neutral to, or contradicts...
Supported / partial / unsupported	verification verdicts for a claim
Misattributed citation	claim is true but cites the wrong passage
Faithfulness	share of the answer's claims that the sources support
Precision@k / Recall@k	share of retrieved chunks that are relevant / share of relevant ...
MRR / nDCG	rank-aware retrieval quality measures
Ablation	removing/adding one component at a time to measure its effe...
LangGraph	library for building stateful LLM workflows as graphs with cond...
Dev / test split	data for tuning vs data for the final, untouched evaluation

---

13. Likely viva questions

Q1. Why not just use a large k for everything?
Large k raises cost and latency and puts distractors in the prompt, which can hurt answers. A small k fails on multi-hop questions. Adaptive k tries to spend context only where needed. Whether it wins is what Experiments 1 and 3 test.

Q2. How do you decide a query is complex without using length?
Through information needs (how many distinct facts), reasoning type (lookup to bridge), whether several sources are needed, lexical cues for comparison/nesting, and the LLM's own label, combined as a weighted
score. A long but single-fact question scores low.

Q3. Why interpolate k instead of three fixed values?
Questions near a label boundary should not jump from 3 to 4 or 7 to 8 abruptly. Interpolation inside the band makes depth follow the score smoothly, while the bands keep to the specified ranges.

Q4. Why decompose and use RRF?
A multi-hop question embedded as one vector tends to retrieve only the most prominent hop. Searching each
sub-question retrieves each hop; RRF merges lists without needing comparable scores, and the coverage guarantee keeps every sub-question represented.

Q5. Why not use cosine similarity to verify claims?
Similarity measures topical closeness, not truth. "Founded in 1999" and "founded in 2016" are almost identical embeddings. Entailment models compare meaning and detect contradiction. We use similarity only
to pick candidate evidence.

Q6. Can the LLM judge hallucinate evidence?
It could, so its verdict is conditional on a verbatim quote that must be found in the named passage. Otherwise the verdict is downgraded to unsupported. Unparseable output also fails closed.

Q7. How do you prevent infinite loops?
`max_retries` (default 2) bounds regeneration, `max_expansions` and `k_cap` bound retrieval, the LangGraph recursion limit is a final backstop, and a best-attempt rule guarantees a result.

Q8. What if the answer is not in the documents?
The prompt tells the model to say so. Abstentions are not treated as claims, so they pass. If the model instead invents something, verification flags it, refinement tries more evidence, and if none exists the unsupported statement is removed and an uncertainty notice is shown.

Q9. Is the evaluation biased toward your system?
It could be if we judged with our own verifier. So the evaluator uses a different NLI model and a stronger different LLM, and a human-labelled sample calibrates how much to trust them. Remaining bias (same provider family, incomplete gold labels, small test sets) is documented as a limitation.

Q10. What does verification cost?
Extra LLM calls (claim extraction, occasional judge), NLI compute, and possible regeneration. Latency and tokens are tracked per stage in every response, and Experiment 5 plots reliability against cost.

Q11. How do you know baseline and proposed are comparable?
They share store, embeddings, chat model, temperature, seed and the generation prompt; one test checks
that the graph with both features off equals the baseline. Differences therefore come from the added components.

Q12. Why LangGraph?
The workflow is a state machine with conditional edges (expand or not, retry or finish), shared state and bounded loops. LangGraph expresses that explicitly and streams per-node progress to the UI.

Q13. What would you do next?
Calibrate thresholds on larger dev data, train a small classifier from logged complexity labels,
try a stronger NLI model for numeric claims, and evaluate on a real domain corpus with human-written answers.

Q14. Can the claim "adaptive RAG is better" be made now?

No. It can be made only if the held-out test results, with confidence intervals, show it. If they do not, the report states that.